

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Analyse de données appliquée & Business Intelligence

M05_C01_le_langage_commun_des_transformations.md · document de production

Table des matières

MODULE M05.C01 — LE LANGAGE COMMUN DES TRANSFORMATIONS : UN FICHIER, TROIS MOTEURS, UN VERDICT	3
1. Objectifs du chapitre	3
2. Pourquoi cette notion est importante	4
3. Explication simple	4
4. Vocabulaire essentiel	5
5. Cours approfondi — la table des 10 opérations dans les 4 outils	6
5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoubler → réparer → fusionner → agréger → exporter	8
5.2 Le pivot et le dépivot : la même idée, deux gestes	8
6. Exemple concret — la coupe de mars 2025 refaite trois fois	9
6.1 Power Query (Excel Microsoft 365) — recette	9
6.2 SQL (DuckDB 1,5,5) — script exécutable	10
6.3 pandas 2,2,3 (Python 3,13) — notebook	11
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	12
7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)	12
7.2 L'empreinte sha256 (la mesure qui ne ment pas)	12
7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)	12
7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)	13
7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)	13
8. Erreurs fréquentes	13
9. Bonnes pratiques professionnelles	14
10. Exercice guidé — la coupe d'avril 2025 dans les 3 outils (35 min, /10)	15
11. Exercices autonomes	16
12. Correction détaillée	16
13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)	17
14. Boîte à outils du chapitre	17
15. Résumé du chapitre	18
16. À retenir	18
17. Évaluation formative (auto-correction, 8 min)	18

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Outils de ce chapitre : Power Query (Microsoft 365), SQL (DuckDB 1,5,5), pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Une transformation — convertir un montant en nombre, dédoubler une clé métier, fusionner un client et une remise — n'a pas trois noms et trois syntaxes : elle a **un geste et trois écritures**. Le geste est invariant (sélectionner, convertir, nettoyer, filtrer, trier, dédoubler, calculer, joindre, agréger, pivoter) ; les écritures diffèrent (formule Power Query, expression SQL, méthode pandas). Le chapitre installe la table des 10 opérations dans les 4 outils (Power Query — SQL — pandas — Excel), et la preuve par la mesure que **les trois moteurs, sur le même fichier, donnent la même table au centime près** : c'est la règle d'or du module.

Base de travail — le fil rouge du module, mars 2025. Un seul fichier, le `ventes_brutes.csv` livré (243 360 lignes au total, 28 516 920 octets, 20 colonnes), coupé à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc de 464 096 003 FCFA** (avant dédoublonnage et réparation, mesure `m05_filrouge_mars_*`). Cette coupe est la **première opération du module** (l'opération « importer + filtrer ») et elle est refaite trois fois dans ce chapitre — une fois dans chaque moteur — pour montrer que la même intention produit la même table. Les **deux chiffres clés** qui reviennent dans les autres chapitres : **mars = 6,49 %** de la fenêtre du projet (120 000 lignes, total 7 145 910 735 FCFA, mesure `m05_fenetre_total_ttc`) et **la jointure naïve sur `id_client` seul = 3 978 lignes** ($\approx 39 \times$ l'enrichissement juste, sans aucune erreur affichée — c'est la leçon du module, mesure `m05_filrouge_mars_jointure_naive`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **nommer** les 10 opérations de la préparation dans les 4 outils (Power Query, SQL, pandas, Excel) — la table du §5 est l'unique référence du module ;
- **exécuter** la première opération (couper un fichier à la date, importer, filtrer) dans chacun des 3 moteurs sur le même fichier, et vérifier que les 3 résultats ont la **même empreinte** ;
- **convertir** un montant en texte (espace-milliers, suffixe « FCFA ») en entier sans perte, en doublant la conversion d'un compteur d'échec (`errors="coerce" + isna().sum()` en pandas, `TRY_CAST + COUNT(*) WHERE ... IS NULL` en SQL, `Value.Try + Errors` en Power Query) ;
- **dédoubler** une table selon une clé métier — pas seulement une clé primaire — et expliquer pourquoi la clé `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure` est la bonne clé sur ce fichier (les 6 répétitions intra-ticket légitimes y sont préservées, mesure `m05_repetitions_legitimes_brut`) ;
- **réparer** les 945 dates à mois et jour échangés qui sont dans le brut livré (résidu de l'histoire M04, plan M05 §1.4) par une transposition jour/mois détectable par `mois(date_vente) ≠ mois` ;
- **rejouer** le même pipeline sur avril et mai 2025 (6 639 et 5 530 lignes, mesures `m05_filrouge_avril_lignes` et `m05_filrouge_mai_lignes`) pour prouver qu'il n'est pas l'œuvre d'un fichier mais d'une méthode ;
- **estimer** le passage à un million de lignes en extrapolant les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles — sans publier cette extrapolation comme une mesure.

2. Pourquoi cette notion est importante

- Vous recevrez le fichier d'un autre, dans un format qui n'est pas celui que vous utilisez. La question n'est pas « quel outil choisir » mais « comment exécuter la même transformation dans l'outil qui est devant vous ». M05 est le module où vous apprenez à **porter** un geste d'un outil à l'autre, pas à en utiliser un seul.
- Sur le fil rouge mars 2025, additionner la colonne des montants **tels qu'ils arrivent** rend une somme **strictement inférieure** au 464 096 003 FCFA de référence (mesure `m05_filrouge_mars_total_ttc`) — les montants texte ne s'additionnent pas, et le moteur tolérant (`pandas.read_csv` par défaut, `read_csv_auto` de DuckDB, ou l'import Power Query sans `TransformColumnTypes`) avale le défaut en silence. La **quantité perdue** dépend du moteur : `pandas.read_csv` met `NaN` et la somme saute la ligne ; DuckDB `read_csv_auto` met `0` et la ligne compte pour zéro ; Power Query met `Error` et la ligne est retirée si on coche « Supprimer les erreurs ». Le **chiffre exact** dépend du moteur — c'est pourquoi la conversion est la première transformation à apprendre, et **son compteur d'échec** est la première discipline à installer.
- À l'inverse, additionner après une conversion **sans dédoubler** rend 484 022 763 FCFA (4,3 % de trop) : les 86 doublons exacts de mars (sur 3 360 injectés dans le brut, mesure `m05_filrouge_mars_doublons`) sont comptés deux fois. La leçon M04 (« un défaut qu'on ne voit pas dans le bon sens produit une erreur plus grosse que le défaut qu'on croyait corriger ») **se répète ici sur les chiffres qui vont au mois**.
- Le tableau de bord du magasin — celui que M03 a appris à construire au tableur — **consomme cette table propre**. Si la table est fausse, le tableau est faux, et la décision qui suit est fausse. M05 est le moment où la qualité devient reproductible : Power Query la rejoue à l'ouverture du classeur, SQL la rejoue à chaque chargement, pandas la rejoue à chaque exécution du notebook.
- L'enrichissement par les remises (mesuré dans la jointure naïve à 3 978 lignes) est l'erreur la plus coûteuse du métier : **39 fois trop, sans message d'erreur**. C'est la jointure à la clé complète — `id_client × annee × mois` — qui la referme, et c'est cette jointure qui est enseignée ici puis exécutée dans les trois outils.
- Enfin, **les trois moteurs ne s'additionnent pas** : un même nettoyage doit être **rejouable** dans chacun, et le verdict — même total, mêmes lignes — est ce qui rend le travail transmissible. La fiche de contrôle de ce module est l'**empreinte sha256** de la table triée : c'est l'objet du §7.

3. Explication simple

On ne prépare pas un fichier dans un outil : on **exécute un pipeline** qui, dans n'importe quel outil, produit la même table. La preuve par l'exemple : prenez mars 2025, exécutez les mêmes 10 opérations dans Power Query, dans SQL et dans pandas — les trois fichiers de sortie ont la même empreinte.

Outil	Ce que vous tapez		Ce que vous obtenez
Power (Excel)	Query	10 étapes dans le panneau de droite	une requête <code>mars2025</code> , actualisable à l'ouverture
SQL (1,5,5)	(DuckDB	<code>SELECT ... FROM read_csv(...) WHERE date_vente BETWEEN ...</code>	une vue <code>mars2025</code> , recalculable à chaque session
pandas (3,13)	(Python	<code>pd.read_csv(...)</code> <code>df[df.date_vente.between(...)]</code>	puis un dataframe, reproductible en réexécutant le notebook

Le **geste** est le même (importer, filtrer à la date, convertir, dédoubler, réparer, nettoyer, joindre, calculer, agréger, pivoter). Les **écritures** diffèrent (formule Power Query, expression

SQL, méthode pandas, formule Excel). Ce chapitre installe la table des correspondances, puis la démontre par la mesure : **trois fichiers de sortie, une seule table**.

La dernière section (le verdict, §7) **mesure** l'égalité : mêmes lignes (6 884), même total `montant_ttc` (464 096 003 FCFA), même empreinte sha256 — et la tient pour acquise seulement après que les trois pipelines ont effectivement tourné. L'extrapolation au million de lignes (mesure non disponible : étiquetée comme telle) vient en germe ici et se referme dans C05.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Transformation — Transformation	Toute opération qui modifie une colonne ou une table : conversion de type, nettoyage de texte, filtre, tri, dédoublonnage, colonne calculée, jointure, agrégation, pivot/dépivot.	Confondre transformation et chargement : le chargement (Power Query = « Fermer et charger », SQL = CREATE TABLE AS, pandas = <code>to_csv</code>) est la fin du pipeline, pas une transformation.
Source — Source	Le fichier ou la table d'où partent les données : CSV, XLSX, vue SQL, dataframe pandas.	Ouvrir un CSV « à la main » dans Excel puis l'enregistrer en <code>.xlsx</code> : la source est perdue, le re-jeu est impossible.
Étape (Power Query) — Step	Une transformation Power Query est une suite d'étapes nommées dans le panneau de droite ; chaque étape est une fonction <code>Table.X(args)</code> appliquée à la précédente.	Ajouter une étape sans la nommer (« Étape appliquée ») : c'est illisible et c'est ce qui rend la requête non transmissible.
Expression SQL — expression SQL	Une requête SQL (SELECT, WITH, CREATE TABLE AS, INSERT...SELECT) appliquée à une source DuckDB.	Citer un <code>id_vente</code> dans un SELECT et l'oublier dans le GROUP BY : la requête tourne, le résultat est faux — DuckDB ne dit rien.
Méthode pandas — pandas method	Toute fonction membre d'un dataframe — DataFrame en anglais, structure tabulaire à deux dimensions étiquetée par lignes et colonnes — : <code>df.assign</code> , <code>df.pipe</code> , <code>df.merge</code> , <code>df.groupby</code> , <code>df.pivot_table</code> .	Enchaîner des affectations <code>df = df.fillna(...)</code> qui réécrivent <code>df</code> sans le dupliquer : une erreur plus haut devient introuvable.
Jointure — Join	Combinaison de deux tables selon une ou plusieurs colonnes communes. La clé complète (<code>id_client</code> × <code>annee</code> × <code>mois</code>) renvoie 1 714 lignes enrichies sur la fenêtre du projet ; la clé partielle (<code>id_client</code> seul) en renvoie 41 716 (mesure <code>m05_fenetre_jointure_naive</code> , ≈ 24 ×) — c'est l'erreur la plus coûteuse du module.	Faire INNER JOIN par défaut : les 247 clients inconnus de la fenêtre disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite.	Oublier le <code>_type</code> : on ne sait pas quelles lignes ont été enrichies vs celles qui ne l'ont pas été.
Clé métier — Business key	Combinaison de colonnes qui, dans le monde, identifie une ligne (sur ce fichier : <code>id_ticket</code> , <code>id_produit</code> , <code>quantite</code> , <code>montant_ttc</code> , <code>heure</code> — la 5-clé qui préserve les 6 répétitions intra-ticket légitimes).	Utiliser <code>id_vente</code> comme clé : il est unique mais il ne dit rien sur l'affaire ; les 3 360 ressaisies ont des <code>id_vente</code> neufs (mesure <code>m05_ressaisies_brut_id_vente_min</code> = 240 001, <code>m05_ressaisies_brut_id_vente_max</code> = 243 360).
Dédoublonnage — Deduplication	Retrait des lignes dont la clé métier revient deux fois.	dédoubler sur la position (ligne 47 et ligne 142 sont « la même ») : c'est ce que le brut livré a fait (les 3 360 doublons sont tous en fin de fichier, <code>id_vente</code> 240 001 → 243 360).
Colonne calculée — Calculated column	Nouvelle colonne dérivée des autres (<code>quantité</code> × <code>prix unitaire</code> , <code>montant_ttc</code> = <code>montant_ht</code> + <code>montant_tva</code> , date avec transposition).	Calculer dans la vue sans la matérialiser : à chaque rechargement, on recalcule, et l'empreinte change.
Empreinte (hash) — Hash fingerprint	Somme sha256 des lignes triées au format canonique. C'est le contrôle que les trois moteurs produisent la même table.	Hasher les lignes dans leur ordre de lecture : deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Oublier la graine (seed=42) sur une étape aléatoire : deux exécutions, deux tables.
Coupe à la date — Date filter	Sélection des lignes dont la date est dans une fenêtre (ici 2025-03-01 → 2025-03-31).	Faire la coupe sur la date de saisie plutôt que la date de vente : on retire les ventes du week-end précédent saisies le lundi.
Transposition (jour/mois) — Date swap	Jour et mois échangés dans une date (ex. 2025-03-07 lu comme 2025-07-03). Sur ce fichier : 945 lignes au total, 493 dans la fenêtre, toutes réparables par échange jour/mois (l'année est cohérente sur 945/945, mesure m05_dates_transposees_pures).	Détecter par le format (/ vs -) : le fichier est en ISO, la transposition est invisible au format — c'est la colonne mois recalculée qui la trahit.
Pipeline — Pipeline	Suite ordonnée de transformations. La même liste, exécutée dans trois outils, produit la même table.	Écrire le pipeline dans l'urgence, sans nommer les étapes : la transmission à un collègue prend une journée.

5. Cours approfondi — la table des 10 opérations dans les 4 outils

La table ci-dessous est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà. Les 10 opérations sont numérotées ; les 2 opérations-cadres (« importer » et « exporter / charger ») sont traitées dans chaque chapitre d'outil, parce qu'elles diffèrent trop pour tenir en une colonne.

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pano
1	Sélectionner / renommer des colonnes	<code>= Table.SelectColumns(src, {"id_vente", "date_vente", "montant_ttc"})</code> puis <code>= Table.RenameColumns(...)</code>	<code>SELECT id_vente, date_vente, montant_ttc FROM read_csv('ventes_brutes.csv')</code>	df =
2	Convertir le type	<code>= Table.TransformColumnTypes(src, [{"montant_ttc", type number}])</code>	<code>TRY_CAST(montant_ttc AS BIGINT) + COUNT(*) WHERE ... IS NULL</code> pour le compteur d'échec	pd.t
3	Nettoyer le texte (espaces, suffixes)	<code>= Table.ReplaceValue(src, " FCFA", "", Replacer.ReplaceText, {"montant_ttc"})</code> puis <code>= Table.Trim</code>	<code>TRIM(REPLACE(montant_ttc, ' FCFA', ''))</code>	df["
4	Filtrer les lignes (la coupe à la date)	<code>= Table.SelectRows(src, each [date_vente] >= #date(2025,3,1) and [date_vente] <= #date(2025,3,31))</code>	<code>WHERE date_vente BETWEEN '2025-03-01' AND '2025-03-31'</code>	df[d
5	Trier	<code>= Table.Sort(src, [{"date_vente", Order.Ascending}, {"id_vente", Order.Ascending}])</code>	<code>ORDER BY date_vente, id_vente</code>	df.s
6	Dédoubler (clé métier)	<code>= Table.Distinct(Table.SelectColumns(src, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}))</code>	<code>SELECT DISTINCT ON (id_ticket, id_produit, quantite, montant_ttc, heure) ... ORDER BY id_vente (PostgreSQL) ; en DuckDB : SELECT * EXCLUDE rn FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY id_ticket, id_produit, quantite, montant_ttc, heure ORDER BY id_vente) AS rn FROM src) WHERE rn = 1</code>	df.s keep

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pandas
7	Colonne calculée	<code>= Table.AddColumn(src, "ca_ht", each [quantite] * [prix_unitaire_ht] * (1 - [taux_remise]), type number)</code>	<code>SELECT *, quantite * prix_unitaire_ht * (1 - taux_remise) AS ca_ht FROM src</code>	df.a
8	Fusionner (jointure)	<code>= Table.NestedJoin(gauche, "id_client", remises, "id_client", "r", JoinKind.LeftOuter) puis = Table.ExpandTableColumn</code>	<code>SELECT ... FROM gauche LEFT JOIN remises USING (id_client, annee, mois) (la clé complète, pas id_client seul !)</code>	df.m
9	Agréger	<code>= Table.Group(src, {"id_client", "annee", "mois"}, {"ca", each List.Sum([montant_ttc]), type number})</code>	<code>SELECT id_client, annee, mois, SUM(montant_ttc) AS ca FROM src GROUP BY id_client, annee, mois</code>	df.g
10	Pivoter / dépivoter	<code>= Table.Pivot(src, List.Distinct(src[id_produit]), "id_produit", "montant_ttc", List.Sum) (pivot) / = Table.UnpivotOtherColumns(...) (dépivot)</code>	<code>SELECT id_client, SUM(CASE WHEN id_produit = 'P1' THEN montant_ttc ELSE 0 END) AS P1 ... FROM src GROUP BY id_client (pivot) ; ou UNPIVOT (dépivot, PostgreSQL ; DuckDB : SELECT id_client, col, val FROM src UNPIVOT(val FOR col IN (P1, P2, ...)))</code>	df.p fill valu

Opérations-cadres (hors table, traitées par chapitre d'outil) :

Cadre	Power Query	SQL	pandas
Importer	<code>= Csv.Document(File.Contents("..."), [Delimiter=";", Encoding=65001])</code>	<code>CREATE TABLE ventes AS SELECT * FROM read_csv_auto('ventes_brutes.csv')</code>	<code>pd.read_csv("ventes_brutes.csv", encoding="utf-8-sig", dtype=str)</code>
Charger / exporter	<code>= ... & "Fermer et charger", ou « Fermer et charger dans... » → connexion uniquement</code>	<code>COPY (SELECT * FROM mars2025) TO 'mars2025.parquet'</code>	<code>df.to_parquet("mars2025.parquet") ou df.to_csv("mars2025.csv", index=False)</code>

Pourquoi 10 + 2 = 12, et pas 12 directement ? Les deux opérations-cadres (importer, charger) sont **plus spécifiques à l'outil** que les 10 opérations internes : l'import Power Query se fait via une fonction M qui détecte le schéma, l'import SQL par `read_csv_auto`, l'import pandas par `read_csv` ; ces trois écritures sont suffisamment différentes pour justifier un traitement par outil (chapitres C02, C03, C04). Les 10 opérations internes, elles, ont un **geste identique** d'un outil à l'autre — c'est la table ci-dessus qui le montre.

Définition. Une **transformation** est une opération qui modifie une colonne ou une table sans changer la source. Une étape Power Query est une fonction `Table.X(args)` appliquée à la précédente ; une requête SQL est une expression `SELECT ... FROM ... WHERE ...` qui prend une table et en rend une autre ; une méthode pandas est une fonction membre d'un dataframe qui prend une table et en rend une autre. Les trois formes sont **équivalentes** : mêmes entrées, même sortie, mêmes précautions.

À retenir. Les 10 opérations sont un **geste**, pas une **syntaxe**. L'ordre a un sens : on importe (1), on sélectionne les colonnes utiles (1), on convertit les types (2) avant de filtrer (4) — sinon le filtre compare des chaînes « 1 200 FCFA » à 1 200, et la comparaison échoue. On dédoublonne (6) avant la jointure (8) — sinon les 35 couples de remises en double explosent en 41 716 lignes (mesure `m05_fenetre_jointure_naive`). On convertit (2) et on nettoie (3) avant d'agréger (9) — sinon la somme saute les lignes texte.

Définition. Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une seule dans la table. Sur le fil rouge mars, `id_vente` est une **clé candidate** (6 884 valeurs distinctes sur 6 884 lignes) mais **ne dit rien de l'affaire** : un client a pu acheter le même produit deux fois le même jour, avec deux `id_vente` distincts. La **clé métier** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'autre clé candidate — elle dit l'affaire, et c'est elle qui dédoublonne.

Définition. Un **compteur d'échec** est un nombre qui dit combien de lignes ont été perdues (ou marquées en erreur) par une conversion. C'est la **différence** entre une conversion traçable (je sais que 114 lignes ont été perdues) et une conversion silencieuse (j'ai additionné, mais je ne sais pas ce qui manque). En pandas : `df["col"].isna().sum() - (df["col_brut"].str.strip() == "").sum()`. En SQL : `COUNT(*) WHERE TRY_CAST(col AS BIGINT) IS NULL AND col IS NOT NULL`. En Power Query : une étape `Table.RowCount(Table.SelectRowsWithErrors(Conversion))`.

5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner → réparer → fusionner → agréger → exporter

C'est l'ordre dans lequel le pipeline est robuste. L'inverser (ex. dédoublonner avant de convertir) ne casse pas tout, mais crée des cas où le moteur tolérant avale un défaut que le moteur strict aurait signalé : par exemple, dédoublonner sur la clé 5-colonnes `id_ticket × id_produit × quantite × montant_ttc × heure` avant d'avoir nettoyé `montant_ttc` (« 1 200 FCFA » et 1200 ne sont pas la même valeur pour `DISTINCT`) **laisse passer** les 86 doublons de mars ; le dédoublonnage après conversion les attrape. Cet ordre est exécuté trois fois dans ce chapitre, dans trois outils, sur le même fichier ; il devient la signature du module.

5.2 Le pivot et le dépivot : la même idée, deux gestes

Pivoter (étalement en colonnes) et dépivoter (replier en lignes) sont **inverses** : si vous avez une ligne par client avec les colonnes `janvier`, `février`, ..., `décembre`, dépivoter rend une ligne par (client × mois) ; pivoter sur le résultat redonne la table d'origine. La tentation est de pivoter à la main dans Excel (TCD) : c'est rapide mais c'est **non reproductible** ; en SQL ou en pandas, le pivot se rejoue. Sur le fil rouge mars, le pivot n'est pas utilisé — c'est un geste plus tard (le tableau de bord par catégorie en C05) — mais il est installé ici parce qu'il complète la table des 10 opérations.

Conseil professionnel. Apprenez les 10 opérations sur un fichier qui n'est pas le vôtre : un CSV public de 5 000 lignes, un export de votre banque, le journal de votre serveur web. Les 10 opérations sont un alphabet : ce qui compte, c'est de les avoir pratiquées une fois dans chacun des 3 moteurs, sur un fichier que vous pouvez recharger sans demander de permission.

6. Exemple concret — la coupe de mars 2025 refaite trois fois

Le **même** geste (importer, couper à la date, convertir, dédoubler), exécuté dans les trois moteurs. Les trois sorties sont sérialisées en `mars2025.csv`, `mars2025.parquet`, `mars2025.xlsx`, puis comparées ligne-à-ligne et colonne-par-colonne.

6.1 Power Query (Excel Microsoft 365) — recette

La requête `mars2025` (à coller dans Power Query → « À partir d'un fichier → À partir d'un texte/CSV ») :

```
let
    Source = Csv.Document(File.Contents("01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
    Entêtes = Table.PromoteHeaders(Source),
    Types = Table.TransformColumnTypes(Entêtes, {"montant_ttc", type text}),
    Nettoyage = Table.TransformColumns(Types, {"montant_ttc", each Text.Trim(Text.Replace(_, "
FCFA", ""))}),
    Conversion = Table.TransformColumnTypes(Nettoyage, {"montant_ttc", type number}),
    FiltreDate = Table.SelectRows(Conversion, each [date_vente] >= #date(2025,3,1) and
[date_vente] <= #date(2025,3,31)),
    Tri = Table.Sort(FiltreDate, {"id_vente", Order.Ascending}),
    Dédoublonnage = Table.Distinct(Table.SelectColumns(Tri,
{"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"})),
    Fusion = Table.NestedJoin(Tri, "id_ticket", Dédoublonnage, "id_ticket", "j",
JoinKind.LeftAnti),
    Gardé = Table.SelectRows(Tri, each [id_vente] <> null and not List.Contains(Fusion[id_vente],
[id_vente])),
    Réparé = Table.TransformColumns(Gardé, {"date_vente", each
        let d = Date.From(_) in
        if Date.Month(d) <> Number.From([mois]) then
            #date(Date.Year(d), Date.Day(d), Date.Month(d))
        else d}),
    Chargé = Réparé
in
    Chargé
```

La sortie attendue : 6 884 lignes (avant dédoublonnage), total `montant_ttc` = 464 096 003 FCFA (`m05_filrouge_mars_total_ttc`). L'étape `Dédoublonnage` construit la clé métier ; l'étape `Fusion` + `Gardé` retire les lignes dont l'`id_ticket` apparaît dans la clé métier dédoublonnée — c'est l'écriture Power Query de `drop_duplicates(..., keep="first")`. L'étape `Réparé` est la transposition jour/mois : si `Date.Month(d) ≠ [mois]`, on échange jour et mois.

6.2 SQL (DuckDB 1,5,5) — script exécutable

```
-- session en mémoire (pas de fichier .duckdb persisté — règle §1.3 du plan M05)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

CREATE OR REPLACE TABLE mars2025 AS
WITH src AS (
    SELECT
        CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS BIGINT) AS montant_ttc_n,
        *
    FROM ventes_brutes
    WHERE montant_ttc IS NOT NULL -- compteurs d'échec
),
coupe AS (
    SELECT * FROM src
    WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31'
),
cle AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
            ORDER BY id_vente
        ) AS rn
    FROM coupe
),
dedup AS (
    SELECT * EXCLUDE rn FROM cle WHERE rn = 1
),
repare AS (
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM date_vente) <> mois
            THEN STRPTIME(date_vente, '%Y-%m-%d')::DATE
                + INTERVAL (EXTRACT(DAY FROM date_vente) - EXTRACT(MONTH FROM date_vente)) DAY
            ELSE date_vente
        END AS date_vente_reparee,
        * EXCLUDE (date_vente)
    FROM dedup
)
SELECT * EXCLUDE rn FROM repare;
```

La sortie attendue : 6 884 lignes, total 464 096 003 FCFA. Le compteur d'échec : `SELECT COUNT(*) FROM ventes_brutes WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '' ) AS BIGINT) IS NULL AND montant_ttc IS NOT NULL` — sur mars 2025, ce compteur rend 114 (mesure m05_filrouge_mars_montants_texte).

6.3 pandas 2,2,3 (Python 3,13) — notebook

```
# bloc publié dans le notebook M05_C01_fil_rouge.ipynb
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, encoding="utf-8-sig", keep_default_na=False)

# (2) conversion, avec compteur d'échec
df["montant_ttc_num"] = pd.to_numeric(
    df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
    errors="coerce",
)
echec_conversion = int(df["montant_ttc_num"].isna().sum() - (df["montant_ttc"].str.strip() == "").sum())
assert echec_conversion == 114, f"compteur d'échec : {echec_conversion} ≠ 114"

# (4) filtre à la date
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884"

# (6) dédoublonnage sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_num", "heure"]
mars = mars.sort_values("id_vente", kind="mergesort").drop_duplicates(CLE, keep="first")

# (7) colonne calculée : ca_ht = quantite * prix_unitaire_ht * (1 - taux_remise)
mars = mars.assign(
    ca_ht=lambda x: pd.to_numeric(x["quantite"], errors="coerce")
    * pd.to_numeric(x["prix_unitaire_ht"], errors="coerce")
    * (1 - pd.to_numeric(x["taux_remise"], errors="coerce"))
)

# (5) réparation des dates transposées (493 dans la fenêtre, mesure m05_dates_transposees_brut)
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# verdict
assert mars["montant_ttc_num"].sum() == 464_096_003, f"total : {mars['montant_ttc_num'].sum()} ≠ 464 096 003"
print("mars :", len(mars), "lignes · total =", mars["montant_ttc_num"].sum(), "FCFA")
```

La sortie attendue : mars : 6 884 lignes · total = 464 096 003 FCFA. Le assert ligne par ligne est la version bloc-notes du contrôleur `controle_python.py` : si le pipeline casse, le notebook s'arrête.

Attention. Les trois écritures ci-dessus **ne sont pas interchangeables** : la conversion SQL `CAST(... AS BIGINT)` est stricte (un texte avec virgule fait échouer toute la ligne) ; `pd.to_numeric(... errors="coerce")` met NaN et continue ; `Power Query TransformColumnTypes(..., type number)` met Error et continue aussi, mais **la ligne est marquée en erreur** (et filtrée si on a coché « Supprimer les erreurs »). Le **compteur d'échec** doit être affiché dans chaque moteur : sans lui, on ne sait pas combien de lignes ont été converties en silence.

Dans les faits. Sur le poste d'un analyste, la même coupe met **47 secondes** en Power Query (actualisation manuelle, Excel Microsoft 365), **0,4 seconde** en SQL DuckDB (en mémoire), **3,1 secondes** en pandas (lecture + parsing). Ces temps ne sont **pas publiés en clé m05_*** : ils varient d'une machine à l'autre, et l'extrapolation au million de lignes (laissant présager ≈ 1 minute en pandas et ≈ 12 secondes en DuckDB) est traitée au C05. Ici, on les note pour mémoire, sans en faire un argument.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que les trois moteurs produisent la même table est la **clé de vôte** du module. Il ne suffit pas que chaque pipeline tourne : il faut que les trois sorties, ligne à ligne et colonne à colonne, soient **identiques au centime près**. La méthode : sérialiser chaque sortie au format canonique (montant_ttc en entier, dates en ISO YYYY-MM-DD, NaN/NULL/null sérialisés en chaîne vide), trier par id_vente, hasher en sha256, et **confronter les trois empreintes**.

7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)

Toutes les colonnes sont **mises au même format** avant la comparaison : montant_ttc en entier sans espace, dates en ISO, NaN/NULL sérialisés en chaîne vide (jamais en NaN ou null), lignes triées par id_vente croissant (et id_vente en int64, pas en chaîne), colonnes jointes par | (un séparateur qui n'apparaît dans aucune cellule du fichier). Ce format est le **contrat** entre les trois moteurs : sans lui, une différence de typage pandas/DuckDB/Power Query devient une différence de table.

7.2 L'empreinte sha256 (la mesure qui ne ment pas)

Le script de référence (qui est la base du contrôleur controle_sql.py et controle_python.py) :

```
# bloc publié (extrait de chiffres_manuel.py – fonction m05())
import hashlib
lignes = []
for r in df.sort_values("id_vente").itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))
empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
```

Sur le dossier projet M05.P (03_exercices/dossier_M05/, recalculé en pandas puis en DuckDB, vérifié par les deux moteurs dans m05()), l'empreinte de la table df_final est 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure m05p_empreinte_sha256). Sur la coupe mars 2025 (fil rouge du présent chapitre), l'empreinte est **différente** — c'est attendu, la table n'a pas les mêmes lignes — mais le **principe** est le même : deux moteurs, deux empreintes, et la règle est empreinte_pandas == empreinte_duckdb.

7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublement)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute (après conversion des 114 textes)	464 096 003 FCFA	m05_filrouge_mars_total_ttc
Montants en texte (« FCFA » + espace-milliers)	114	m05_filrouge_mars_montants_texte
Doublons exacts (avant dédoublement)	86	m05_filrouge_mars_doublons
Remises saisies en points de % (12, 18, 25, 30)	32	m05_filrouge_mars_remises_sup_1
Retours (est_retour = 1)	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète id_client × mois)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Remise par ligne / par couple	11 620 000 / 10 656 000 FCFA	m05_filrouge_mars_remise_par_ligne
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

Définition. Une **empreinte sha256** est une somme cryptographique de 256 bits qui identifie un contenu au bit près. Deux fichiers de 100 Mo ayant la même empreinte sont, **pour toute opération raisonnable**, identiques — c'est la propriété que la commande `sha256sum` utilise pour vérifier un téléchargement. L'empreinte **n'est pas réversible** : on ne retrouve pas la table à partir de l'empreinte, on vérifie seulement l'égalité de deux empreintes.

7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)

Le **même** pipeline, sans aucune modification, exécuté sur avril et mai 2025 :

Mois	Lignes	Total <code>montant_ttc</code> (FCFA)
Mars 2025	6 884	464 096 003
Avril 2025	6 639	(mesure <code>m05_filrouge_avril_lignes</code>)
Mai 2025	5 530	(mesure <code>m05_filrouge_mai_lignes</code>)

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité. C'est cette **portabilité** qui justifie l'investissement du module (5 chapitres, 30 h) : sans elle, la préparation est un script à re-écrire à chaque fichier.

7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)

Les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles (chapitre C05, à venir) extrapolent **de manière linéaire** (à vérifier) à un million de lignes. Cette extrapolation **n'est pas une mesure** : elle est étiquetée comme telle, et la leçon du module (règle 3 du §3 du plan M05) est « une mesure qui tombe en estimation sans le dire est fausse ». Ici, la mesure existe, l'estimation est nommée : le lecteur sait ce qu'il lit.

Attention. Le million de lignes **n'est pas fabriqué** dans cet atelier (règle §4 du plan M05) : aucun fichier d'un million de lignes n'est généré par `dossier_M05.py`. L'extrapolation est faite à partir des **trois tailles réelles** livrées, et le résultat est présenté comme « estimation linéaire, à vérifier sur un fichier plus gros » — pas comme une mesure. C'est la discipline d'honnêteté du module.

8. Erreurs fréquentes

- **Confondre la coupe à la date et le filtre automatique.** La coupe `WHERE date_vente BETWEEN ...` est déterministe ; le filtre automatique Excel peut exclure des lignes masquées par mégarde. Toujours **recharger la source** avant la coupe.
- **Convertir avec `errors="ignore"` (pandas) ou `TRY_CAST` (SQL) sans lire le compteur.** `errors="coerce"` met NaN et continue ; le compteur `isna().sum()` est ce qui dit combien de lignes ont été perdues. Une conversion sans compteur est une conversion sans information — et le contrôleur `controle_python.py` refuse les blocs qui en sont dépourvus.
- **Dédoublonner sur la position** (les lignes 47 et 142 sont « la même ») au lieu de la **clé métier**. Le brut livré a toutes ses ressaisies en fin de fichier (`id_vente` 240 001 → 243 360 — soit 3 360 lignes, mesure `m05_ressaisies_brut_total`, et l'index suit : positions $n-3\,360 \rightarrow n-1$ du

fichier), ce qui rend la position apparemment fiable — mais c'est un artefact du générateur, pas une propriété du réel. La coupe du fichier au 1^{er} janvier 2024 (ce que le projet M05.P demande) met les ressaisies mêlées aux originaux : la position ne dit plus rien.

- **Faire INNER JOIN par défaut.** Les 247 clients inconnus de la fenêtre (mesure `m05_fenetre_clients_inconnus`, ids distincts = 211) **disparaissent** silencieusement, et le total n'est plus celui du fichier d'entrée. La règle du module est `LEFT JOIN` systématique, avec un compteur `WHERE clé_droite IS NULL` pour signaler les non-jointes.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la **clé complète** (`id_client × annee × mois`). La jointure naïve rend **41 716 lignes** sur la fenêtre du projet (`m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste), et **3 978** sur mars 2025 — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans l'étape Fusion.
- **Oublier `keep_default_na=False, dtype=str` à l'import pandas.** Sans ça, pandas devine les types, convertit `id_vente` en `int64` (perdant les `id_vente = 0` qui sont des ventes comptoir), et avale les montants texte sans un mot. La règle est toujours importer en `str`, puis convertir.
- **Réparer les dates transposées par le format (/ vs -).** Le fichier est en ISO ; la transposition est **invisible au format**. C'est la colonne `mois` recalculée (`EXTRACT(MONTH FROM date_vente) <> mois`) qui la trahit. Tenter de la détecter au format est l'erreur classique du débuteur.
- **Hasher les lignes dans l'ordre de lecture.** Deux moteurs qui trient différemment ont deux empreintes, même sur la même table. La règle est trier par `id_vente` avant de hasher — c'est la sérialisation canonique du §7.1.
- **Publier une mesure qui sort d'une étape aléatoire sans graine.** `sample(n=10_000, random_state=...)` en pandas, `USING SAMPLE 10%` en DuckDB. Sans graine, deux exécutions donnent deux mesures. La règle est toute étape aléatoire porte une graine, et la graine est documentée dans le code.
- **Croire que « Power Query est le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel (Microsoft 365) ; sans Excel, il n'existe pas. Le pipeline C02 est une **recette** (règle 7 du §3 du plan M05) : ses points de contrôle sont manuels, et chaque chiffre qu'il affiche est exécuté en SQL et en pandas sur les mêmes données. C'est ce qui rend le module honnête.

À retenir. Les 5 erreurs qui coûtent le plus cher sur le fil rouge : (1) convertir sans compteur (114 montants texte deviennent NaN en silence) ; (2) dédoublonner sur la position (les 86 doublons de mars passent) ; (3) jointure naïve (3 978 lignes au lieu de 101) ; (4) oublier la réparation des dates transposées (493 lignes de la fenêtre sortent au 31/09/2025, qui n'existe pas) ; (5) INNER JOIN par défaut (les 247 inconnus disparaissent, et le total du fichier d'entrée n'est plus reproductible).

9. Bonnes pratiques professionnelles

- **Une étape = un nom, en toutes lettres, dans les trois outils.** Power Query : « Mars2025_Coupe », « Mars2025_Conversion », « Mars2025_Dédoublonnage ». SQL : --
Étape 4 : filtre à la date. pandas : `mars = df[...].copy(), mars_ttc_num = pd.to_numeric(...), mars_dedup = mars.drop_duplicates(...)`. Une étape sans nom est une étape que personne ne peut reprendre.
- **Le compteur d'échec n'est jamais en commentaire.** C'est la première chose que le relecteur regarde. En pandas : `print("conversion échouée :", mars_ttc_num.isna().sum())`. En SQL : `SELECT 'mars_conversion', COUNT(*) FROM ventes_brutes WHERE TRY_CAST(...) IS NULL AND`

`montant_ttc IS NOT NULL.` En Power Query : une étape = `Table.RowCount(Table.SelectRowsWithErrors(Conversion))` qui affiche le résultat.

- **L’empreinte est dans le bloc final, pas dans le README.** C’est le **contrôleur** (`controle_python.py`, `controle_sql.py`) qui la calcule et la publie — pas l’apprenant qui la tape à la main. Si l’empreinte est dans le README, elle n’est plus rejouable ; si elle est dans le contrôleur, elle l’est.
- **Le pipeline est versionné.** Chaque modification du pipeline incrémente une version (`mars2025_v1`, `mars2025_v2`) ; l’empreinte change, le tableau de bord est mis à jour ; **on ne ré-écrit pas un pipeline en place.** C’est la même règle que les versions de code (Git), appliquée aux fichiers de données.
- **Le pipeline documente ses défauts non corrigés.** Sur le fil rouge : 32 remises > 1 (saisies en points de %, à diviser par 100 — règle du C02), 74 retours (laissés en l’état pour la traçabilité), 10 clients inconnus (laissés en `id_client` brut). Ce ne sont pas des défauts à corriger dans le pipeline de coupe : ce sont des **cas à signaler au tableau de bord.**
- **Le verdict est publié en clé `m05_*`.** Tout chiffre qui sort du chapitre (6 884 lignes, 464 096 003 FCFA, 114 textes, 86 doublons, 101 enrichies, 3 978 naïves) provient de `chiffres_manuel.py` (mesures `m05_filrouge_mars_*`). Pas de chiffre « à peu près », pas de chiffre copié d’un corrigé : **on exécute, on lit, on recopie.**

Conseil professionnel. Avant de soumettre un pipeline à un collègue, exécutez-le sur **trois fichiers** de votre portfolio : un petit (5 000 lignes), un moyen (50 000), un gros (500 000). Les bugs n’apparaissent pas sur le fichier de 5 000 lignes — ils apparaissent sur le passage à l’échelle. C’est la même leçon que C05 (« arbitrer un outil par situation ») : un pipeline qui n’a été testé que sur un fichier n’est pas un pipeline, c’est un script.

10. Exercice guidé — la coupe d’avril 2025 dans les 3 outils (35 min, /10)

Objectif. Reprendre le pipeline du §6 sur avril 2025 et vérifier que les 3 moteurs produisent la même table (empreinte sha256 identique).

Énoncé.

1. Importer `01_socle_donnees/data/brut/ventes_brutes.csv` dans **chacun des 3 outils**.
2. Couper à la date `2025-04-01` → `2025-04-30` (filtre, `WHERE`, `between`, ou étape Power Query).
3. Convertir `montant_ttc` en entier (avec compteur d’échec affiché).
4. Dédoublonner sur la clé métier 5-colonnes (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`).
5. Réparer les dates transposées (règle du §7.5).
6. Sérialiser au format canonique (§7.1), trier par `id_vente`, hasher en sha256.
7. Comparer les **trois empreintes** — elles doivent être identiques au bit près.

Barème (/10).

Critère	Points
3 pipelines tournent sans erreur	2
Nombre de lignes affiché dans chaque moteur = 6 639 (mesure <code>m05_filrouge_avril_lignes</code>)	2
Compteur d’échec de conversion affiché (sa valeur n’est pas demandée, sa présence oui)	2
3 empreintes sha256 identiques	4

Critère	Points
Total	10

Dans les faits. Cet exercice prend 25 minutes à un apprenant qui a déjà pratiqué le §6, et 45 minutes à un apprenant qui découvre. La différence est presque entièrement sur l'étape 6 (la sérialisation canonique) : un apprenant qui n'a pas compris pourquoi le format canonique existe perd 20 minutes à comprendre pourquoi ses empreintes ne collent pas.

11. Exercices autonomes

- **Exercice 11.1 (10 min).** Reprendre le pipeline du §6 et **ajouter** l'étape 7 du §5 (la colonne calculée `ca_ht = quantite × prix_unitaire_ht × (1 - taux_remise)`). Vérifier que la colonne est présente dans les 3 sorties et que sa somme est identique au centime près.
- **Exercice 11.2 (15 min).** Reprendre le pipeline du §6 et **remplacer** la coupe `BETWEEN` par une coupe `EXTRACT(MONTH FROM date_vente) = 3 AND EXTRACT(YEAR FROM date_vente) = 2025` (SQL) / `df["date_vente"].dt.month == 3` (pandas) / `each Date.Month([date_vente]) = 3` (Power Query). Vérifier que le nombre de lignes est **toujours 6 884**, et que le **total montant_ttc est inchangé**.
- **Exercice 11.3 (30 min).** Le pipeline du §6 produit 6 884 lignes, total 464 096 003 FCFA. **Refaire le pipeline sur les 120 000 premières lignes du fichier** (la fenêtre du projet M05.P). Vérifier : total attendu = **7 145 910 735 FCFA** (mesure `m05_fenetre_total_ttc`), 493 dates transposées réparées (mesure `m05_fenetre_dates_transposees`), 1 714 lignes enrichies (mesure `m05_fenetre_lignes_enrichies`).
- **Exercice 11.4 (45 min, optionnel).** Mesurer le temps d'exécution de chacun des 3 pipelines sur le fichier complet (243 360 lignes). Reporter les temps dans un tableau ; calculer la moyenne et l'écart-type sur 5 exécutions ; **étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** La colonne `ca_ht` apparaît dans les 3 sorties. Sa somme (sur mars 2025) est à mesurer dans le notebook — le module ne publie pas cette clé en `m05_*` : elle dépend du coefficient d'arrondi HT/TTC du fichier brut. L'erreur classique : oublier `errors="coerce"` sur `taux_remise` (pandas met NaN sur les remises en points de %, et le calcul `1 - NaN` rend NaN). La règle de ré-exécution : la valeur mesurée doit coïncider entre pandas et SQL au centime près ; si ce n'est pas le cas, c'est l'arrondi HT/TTC du fichier brut qui est différent, et il faut l'écrire dans le rapport qualité (réinvestissement de M04.P).
- **Exercice 11.2.** Le nombre de lignes est **6 884** dans les deux écritures. Le total `montant_ttc` est **inchangé** à 464 096 003 FCFA. L'erreur classique : oublier l'année dans le filtre (`MONTH = 3` attrape aussi mars 2024 et mars 2026 — sur ce fichier : $6\,884 + 5\,530 = 12\,414$ lignes, ce qui ne correspond pas du tout au 6 884 attendu).
- **Exercice 11.3.** Le total est **7 145 910 735 FCFA**, conforme à `m05_fenetre_total_ttc`. Les **493 dates transposées** sont réparées par l'étape Réparé (Power Query) / `CASE WHEN ... THEN ...` (SQL) / `mars.loc[inco, "date_vente"] = ...` (pandas). Les **1 714 lignes enrichies** sont issues de la jointure `LEFT JOIN` sur la clé complète (`id_client`, `annee`, `mois`) après dédoublonnage des remises (qui passe de 9 942 lignes brutes à 9 907 lignes dédoublonnées, mesure `m05_re-mises_couples_ressaisis` = 35). La **jointure naïve sur `id_client` seul** rend **41 716 lignes**

(mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste) — c'est l'erreur que le projet M05.P fait payer.

- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (pandas 2,2,3, DuckDB 1,5,5, Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query, 0,4 s SQL DuckDB, 3,1 s pandas (sur 6 884 lignes, à multiplier par ≈ 35 pour 243 360 lignes). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)

Énoncé. Reprendre le pipeline du §6 sur le **vrai** fichier du projet M05.P (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes = 120 000 + 1 720 copies injectées). Le **livrable** est triple : (a) une requête Power Query `mars2025_v1`, (b) un script SQL `mars2025.sql`, (c) un notebook `mars2025.ipynb`. Les **trois sorties** sont sérialisées en `mars2025.csv` et **leur empreinte sha256 doit être identique**.

Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé à la date sur les 120 000 premières lignes du fichier).
2. **Total montant_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).
3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'échec de conversion affiché** dans les 3 pipelines (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** entre les 3 sorties (vérifiée par `controle_python.py`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — rejouer le pipeline et vérifier l'empreinte :

1. **La table des 10 opérations** (§5) — l'unique référence du module, à imprimer et à garder sous le coude pendant les chapitres C02, C03, C04.
2. **Le format canonique de sérialisation** (§7.1) — la règle qui rend les empreintes comparables : entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`, colonnes jointes par `|`.
3. **Le script de calcul d'empreinte** (§7.2) — les 8 lignes de pandas (ou le `SELECT ... FROM ... ORDER BY ... USING SAMPLE`) qui transforment une table en sha256. À copier dans `controle_python.py` et `controle_sql.py`.
4. **La clé métier 5-colonnes** (§4, vocabulaire essentiel) — (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) : la clé qui dédouble sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
5. **Le compteur d'échec** (§8) — le réflexe à installer avant chaque conversion : `isna().sum()` en pandas, `TRY_CAST ... IS NULL` en SQL, `Table.SelectRowsWithErrors` en Power Query.

15. Résumé du chapitre

- **Les 10 opérations** de la préparation sont un **geste unique** ; les **écritures** diffèrent selon l'outil (Power Query, SQL, pandas, Excel). La table du §5 est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà.
- **Le pipeline canonique** est : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner (clé métier 5-colonnes) → réparer (transposition jour/mois) → fusionner (clé complète) → agréger → exporter. L'inverser crée des défauts silencieux que le moteur tolérant avale.
- **Le verdict** (« un fichier, trois moteurs, une table ») est l'**empreinte sha256** de la sortie sérialisée au format canonique (entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`). Sans cette sérialisation, deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.
- **Le fil rouge mars 2025** : 6 884 lignes · total 464 096 003 FCFA · 114 montants texte · 86 doublons · 32 remises > 1 · 74 retours · 10 inconnus · 101 enrichies · 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.
- **L'extrapolation au million de lignes** est étiquetée comme telle : c'est une estimation, pas une mesure. Le million de lignes n'est pas fabriqué dans cet atelier.

16. À retenir

À retenir. La préparation d'un fichier n'est pas une tâche Excel, c'est un pipeline reproductible dont le verdict est l'empreinte sha256. Trois disciplines : (1) le **compteur d'échec** à chaque conversion (sans lui, les 114 montants texte de mars deviennent NaN en silence) ; (2) la **clé complète** (`id_client × annee × mois`) à chaque jointure (sans elle, 3 978 lignes au lieu de 101 sur mars, 41 716 au lieu de 1 714 sur la fenêtre) ; (3) la **sérialisation canonique** avant hash (sans elle, deux moteurs qui trient différemment ont deux empreintes). Les 10 opérations sont un alphabet ; les 3 moteurs (Power Query, SQL, pandas) sont trois langues qui disent la même chose.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 10 opérations de la préparation, dans l'ordre canonique. (Réponse : §5.)

Question 2. Sur le fil rouge mars 2025, combien de lignes le pipeline doit-il rendre **avant** dédoublonnage ? **Après** dédoublonnage ? (Réponse : 6 884 avant ; 6 798 après — les 86 doublons de mars sont retirés par la clé 5-colonnes.)

Question 3. Quelle est la différence entre `errors="coerce"` (pandas), `TRY_CAST` (SQL) et `TransformColumnTypes(..., type number)` (Power Query) ? (Réponse : les trois convertissent les textes en nombres en cas d'échec ; pandas met NaN, SQL met NULL (DuckDB : erreur si on n'utilise pas `TRY_CAST`), Power Query met `Error`. Le compteur d'échec est ce qui distingue une conversion traçable d'une conversion silencieuse.)

Question 4. Pourquoi la sérialisation canonique est-elle **avant** le hash sha256 ? (Réponse : sans sérialisation canonique, deux moteurs trient différemment et ont deux empreintes. La sérialisation fixe l'ordre des colonnes, le format des dates, le typage des nombres, et le tri des lignes. C'est le contrat entre les trois moteurs.)

Question 5. Sur le fil rouge, la jointure naïve sur `id_client` seul rend **3 978 lignes** ; la jointure sur la clé complète rend **101 lignes**. Quel est le **rapport** ? Que dit ce rapport du métier ? (Réponse : $3\,978 / 101 \approx 39$. Le rapport dit que les remises sont mensualisées : un client a au plus une remise par mois, et 89 des 101 lignes enrichies correspondent à 89 couples (`client × mois`))

distincts. La jointure naïve « double-compte » chaque ligne de vente par le nombre de mois couverts par le client — c'est l'erreur classique du débutant, et c'est ce que le module refuse.)

Question 6. Citez les 5 erreurs fréquentes qui coûtent le plus cher sur le fil rouge. (Réponse : §8, encadré « À retenir ».)